

C2SafeRust-LLM

Module 1: Input Validation — Line-by-Line Explanation

For Beginners | Mini Project S3

Module 1: Input Validation

Line-by-Line Code Explanation for Beginners

File: src/module1_validation/mod.rs

What does Module 1 do? Module 1 is the **FIRST** stage of the pipeline. Think of it as a strict security guard at the entrance. Before any parsing or analysis begins, it confirms: Is this a real C file? Is it readable? Is it valid text? If anything is wrong, it stops the whole pipeline immediately and reports a clear error message. **One file, one job:** Unlike other modules, Module 1 is a single file (mod.rs) with no sub-files. It produces one output: a `ValidatedCSource` struct or a `ValidationError`.

Imports (Lines 1-3)

```
use std::fmt;
use std::fs;
use std::path::{Path, PathBuf};
```

Line	Import	What it provides
<code>`use std::fmt;`</code>	Standard formatting	Lets us implement the <code>`Display`</code> trait — how errors print as human-readable text.
<code>`use std::fs;`</code>	File system	Provides <code>`fs::read()`</code> , <code>`fs::File::create()`</code> , <code>`fs::remove_file()`</code> — everything needed to read and write files.
<code>`use std::path::{Path, PathBuf};`</code>	Path types	<code>`Path`</code> is a reference to a file path (like a string slice). <code>`PathBuf`</code> is an owned path (like <code>`String`</code>). We accept <code>`Path`</code> as input and store <code>`PathBuf`</code> in the output struct.

Beginner Tip — Path vs PathBuf: Think of `Path` like `&str` (a borrowed view) and `PathBuf` like `String` (owned, can grow). We use `Path` for function parameters (don't need ownership) and `PathBuf` for storing in structs.

The Output Struct: ValidatedCSource (Lines 5-12)

```

/// Output struct produced when a C input file passes validation.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct ValidatedCSource {
    pub file_path: PathBuf,
    pub raw_content: String,
    pub line_count: usize,
    pub byte_size: usize,
}

```

*This is the **SUCCESS** output — what you get back when a file passes all validation checks. Every downstream module (Module 2, Module 3, etc.) uses this struct as its starting point.*

Field	Type	What it stores
<code>`file_path`</code>	<code>`PathBuf`</code>	The full path to the validated C file (e.g., <code>`benchmarks/sample_test.c`</code>).
<code>`raw_content`</code>	<code>`String`</code>	The entire C source code as a UTF-8 string, read into memory.
<code>`line_count`</code>	<code>`usize`</code>	How many lines the file has (counted with <code>`lines().count()`</code>). Shown in Stage 1 output.
<code>`byte_size`</code>	<code>`usize`</code>	How many bytes the file takes on disk (<code>`content.len()`</code>). Shown in Stage 1 output.

About the derives:

Derive	What it gives you
<code>`Debug`</code>	Lets you print the struct with <code>`{:?}`</code> — useful in tests and debug output.
<code>`Clone`</code>	Lets you make an exact copy of the struct with <code>`clone()`</code> . Module 2's preprocessor clones this.
<code>`PartialEq`</code>	Lets you compare two structs with <code>`==`</code> . Used in unit tests.
<code>`Eq`</code>	A stronger form of equality — required alongside <code>`PartialEq`</code> for use in hash maps and sets.

The Error Enum: `ValidationError` (Lines 14-23)

```

/// Errors that can occur during input acceptance and validation.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ValidationError {
    FileNotFound(String),
    InvalidExtension(String),
    FileNotReadable(String),
    InvalidEncoding(String),
    EmptyFile(String),
    PreflightLexicalError(String),
}

```

This is the FAILURE output — what you get when a file fails one of the checks. Each variant holds a `String` with a human-readable description of what went wrong. The `String` inside each variant is the error message.

Variant	When it triggers	Example message
<code>FileNotFound(String)</code>	The file path doesn't exist on disk	<code>"Input file does not exist: 'missing.c'"</code>
<code>InvalidExtension(String)</code>	Extension is not <code>.c</code> or <code>.h</code>	<code>"Expected '.c' or '.h' extension, but found '.txt'"</code>
<code>FileNotReadable(String)</code>	OS permission denied or I/O error	<code>"Failed to read file 'locked.c': permission denied"</code>
<code>InvalidEncoding(String)</code>	File has null bytes (binary) or non-UTF-8	<code>"Input file contains null bytes (binary file detected)."</code>
<code>EmptyFile(String)</code>	File is empty or only whitespace	<code>"Input file 'empty.c' is empty or contains only whitespace."</code>
<code>PreflightLexicalError(String)</code>	Unmatched braces, open strings, etc.	<code>"Unclosed curly brace '{' (1 missing '}'")"</code>

***Why use an enum for errors?** In C, errors are typically just integers (`-1`, `NULL`) with no context. In Rust, we use enums so the caller must **handle every possible error case** — the compiler forces you to. This is one of Rust's key safety features.*

Implementing Display for ValidationError (Lines 25-36)

```
impl fmt::Display for ValidationError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            ValidationError::FileNotFound(msg) =>
                write!(f, "File Not Found Error: {}", msg),
            ValidationError::InvalidExtension(msg) =>
                write!(f, "Invalid Extension Error: {}", msg),
            ValidationError::FileNotReadable(msg) =>
                write!(f, "File Read Error: {}", msg),
            ValidationError::InvalidEncoding(msg) =>
                write!(f, "Invalid Encoding Error: {}", msg),
            ValidationError::EmptyFile(msg) =>
                write!(f, "Empty File Error: {}", msg),
            ValidationError::PreflightLexicalError(msg) =>
                write!(f, "Lexical Validation Error: {}", msg),
        }
    }
}
```

Line	Code	Explanation
<code>impl fmt::Display for ValidationError`</code>	Implementing a trait	This says: "I am teaching `ValidationError` how to display itself as a human-readable string." `Display` is the trait that controls what <code>println!("{}", error)`</code> shows.
<code>fn fmt(&self, f: &mut; fmt::Formatter<'_>) -> fmt::Result`</code>	Method signature	`self` is the enum variant being displayed. `f` is the "formatter" — the destination where text is written. `fmt::Result` is either <code>Ok()</code> (success) or an error.
<code>match self { ... }`</code>	Pattern matching	Checks which variant `self` is. This is like a `switch` statement in C but exhaustive — all variants MUST be covered.
<code>write!(f, "File Not Found Error: {}", msg)`</code>	Write formatted text	<code>write!</code> macro writes text to the formatter `f`. `{}` is where <code>msg`</code> (the String inside the variant) gets inserted.

Why implement Display? With `Display` implemented, you can write `eprintln!("Error: {}", err)` and get a nice message. Without it, you'd only be able to use `{:?}` which gives the raw debug format like `FileNotFound("...")`.

Line 38: Standard Error Trait

```
impl std::error::Error for ValidationError {}
```

Code	Explanation
<code>`impl std::error::Error for ValidationError {}`</code>	The empty <code>`{}`</code> body means we accept all default method implementations. This makes <code>`ValidationError`</code> work with the standard Rust error ecosystem — it can be boxed into <code>`Box`</code> , used with the <code>`?`</code> operator, and handled by error-handling libraries.

The Main Entry Point: `validate_input()` (Lines 40-104)

```
pub fn validate_input(file_path: &Path) -> Result<ValidatedCSource, ValidationError> {
```

Part	Explanation
<code>`pub fn`</code>	This function is public — callable from other modules (like <code>`main.rs`</code>).
<code>`validate_input`</code>	The function name. Called as <code>`module1_validation::validate_input(path)`</code> .
<code>`file_path: &Path;`</code>	Takes a REFERENCE to a path (borrows it, doesn't own it). The caller still owns the path.
<code>`-> Result`</code>	Returns either <code>`Ok(ValidatedCSource)`</code> on success, or <code>`Err(ValidationError)`</code> on failure. This is Rust's way of making errors first-class values instead of exceptions.

Validation Step 1: Check File Existence (Lines 42-48)

```
if !file_path.exists() {
    return Err(ValidationError::FileNotFound(format!(
        "Input file does not exist: '{:?}'",
        file_path
    )));
}
```

Line	Code	Explanation
<code>`if !file_path.exists()`</code>	Existence check	<code>`exists()`</code> is a method on <code>`Path`</code> that calls the OS to check if the path exists. The <code>`!`</code> means "if NOT exists".
<code>`return Err(...)`</code>	Early return	If the file doesn't exist, we immediately stop and return an <code>`Err`</code> . Everything after this only runs if the file exists.
<code>`ValidationError::FileNotFound(format!(...))`</code>	Create error	Constructs the <code>`FileNotFound`</code> variant with a formatted string message.
<code>`format!("{}", file_path)`</code>	Format macro	Like <code>`sprintf`</code> in C — creates a <code>`String`</code> with the path inserted where <code>`{:?}`</code> is. <code>`{:?}`</code> uses the Debug format for paths.

Validation Step 2: Check File Extension (Lines 50-63)

```
if let Some(ext) = file_path.extension() {
    let ext_str = ext.to_string_lossy().to_lowercase();
    if ext_str != "c" && ext_str != "h" {
        return Err(ValidationError::InvalidExtension(format!(
            "Expected '.c' or '.h' extension, but found '{}'",
            ext_str
        )));
    }
} else {
    return Err(ValidationError::InvalidExtension(
        "Input file has no extension; expected a '.c' or '.h' file.".to_string(),
    ));
}
```

Line	Code	Explanation
<code>`if let Some(ext) = file_path.extension()``</code>	Pattern matching on Option	<code>`.extension()``</code> returns <code>`Option<OsStr>`</code> — <code>`Some(ext)``</code> if there's an extension, <code>`None``</code> if not. <code>`if let Some(ext) = ...``</code> is a shorthand to extract the <code>`Some``</code> value.
<code>`ext.to_string_lossy()``</code>	OS string conversion	File extensions are <code>`OsStr``</code> (OS-native strings) which might not be UTF-8. <code>`.to_string_lossy()``</code> converts to a Rust string, replacing invalid chars with <code>`?``</code> .
<code>`.to_lowercase()``</code>	Case normalization	Ensures <code>`.C``</code> and <code>`.c``</code> are both accepted (makes comparison case-insensitive).
<code>`if ext_str != "c" && ext_str != "h"``</code>	Extension check	Only <code>`.c``</code> and <code>`.h``</code> files are valid C source files.
<code>`else { return Err(...) }``</code>	No extension case	If <code>`.extension()``</code> returned <code>`None``</code> , the file has no extension at all — also an error.

Validation Step 3: Read Raw Bytes (Lines 65-75)

```
let bytes = fs::read(file_path).map_err(|err| {
    ValidationError::FileNotReadable(format!(
        "Failed to read file '{}': {}", file_path, err
    ))
})?;

if bytes.contains(&0) {
    return Err(ValidationError::InvalidEncoding(
        "Input file contains null bytes (binary file detected)".to_string(),
    ));
}
```

	Explanation			
	Reads the entire file into a <code>Vec</code> (a list of raw bytes). Returns <code>Result, io::Error></code> .			
	<code>...)</code>	Error conversion	If reading fails, converts the standard <code>io::Error</code> into our custom <code>ValidationError::FileNotReadable</code> . The <code>\</code>	<code>err\</code>
	If <code>.map_err(...)</code> returned an <code>Err</code> , the <code>?</code> immediately returns that error from <code>validate_input</code> . This is how Rust propagates errors without <code>try/catch</code> .			
	Checks if ANY byte in the file is <code>0x00</code> (null byte). Real C source files are text and should never have null bytes. ELF binaries, images, etc. do.			

Validation Step 4: UTF-8 Encoding Check (Lines 77-82)

```
let content = String::from_utf8(bytes).map_err(|_| {
    ValidationError::InvalidEncoding(
        "Input file is not valid UTF-8 text.".to_string(),
    )
})?;
```

	Code	Explanation		
<code>bytes)</code>	Convert bytes to string	Takes the <code>Vec</code> and tries to interpret it as a valid UTF-8 string. Returns <code>Result</code> .		
	<code>_</code>	<code>)</code>	Discard inner error	The <code>_</code> means we don't care about the details of the error, we just want our own message.
	Propagate error	If it's not valid UTF-8, return <code>Err(ValidationError::InvalidEncoding(...))</code> .		

Validation Step 5: Non-Empty Content Check (Lines 84-90)

```
if content.trim().is_empty() {
    return Err(ValidationError::EmptyFile(format!(
        "Input file '{:?}' is empty or contains only whitespace.",
        file_path
    )));
}
```

Line	Code	Explanation
<code>`content.trim()`</code>	Strip whitespace	Removes leading and trailing whitespace (spaces, newlines, tabs).
<code>`.is_empty()`</code>	Empty check	Returns <code>`true`</code> if the trimmed string has zero characters.
Combined	Effect	If the file is completely blank (or just newlines and spaces), this fails. We can't parse an empty file!

Validation Step 6: Pre-flight Lexical Check (Lines 92-93)

```
perform_preflight_lexical_checks(&content)?;
```

Code	Explanation
<code>`perform_preflight_lexical_checks(&content);`</code>	Calls the private helper function (defined later in the same file) that checks for balanced braces, open strings, etc. Returns <code>`Result<(), ValidationError>`</code> .
<code>`?`</code>	If the checks fail, propagate the <code>`ValidationError::PreflightLexicalError`</code> immediately.

Building and Returning the Success Value (Lines 95-103)

```
let line_count = content.lines().count();
let byte_size = content.len();

Ok(ValidatedCSource {
    file_path: file_path.to_path_buf(),
    raw_content: content,
    line_count,
    byte_size,
})
```

Line	Code	Explanation
<code>`content.lines().count()`</code>	Count lines	<code>`.lines()`</code> is an iterator over each line. <code>`.count()`</code> consumes the iterator and counts how many lines there are.
<code>`content.len()`</code>	Byte count	Returns the number of bytes in the UTF-8 string. For pure ASCII C files, this equals the number of characters.
<code>`file_path.to_path_buf()`</code>	Path conversion	Converts the borrowed <code>`&Path;`</code> reference into an owned <code>`PathBuf`</code> so we can store it in the struct.
<code>`Ok(ValidatedCSource { ... })`</code>	Return success	Wraps the struct in <code>`Ok(...)`</code> to signal success. The caller can then pattern-match on <code>`Ok(validated)`</code> .

Line	Code	Explanation
<code>`line_count,` (shorthand)</code>	Field shorthand	Since the variable name matches the field name, Rust lets you write <code>`line_count,`</code> instead of <code>`line_count: line_count,`</code> .

The Preflight Lexical Checker (Lines 106-238)

```
fn perform_preflight_lexical_checks(content: &str) -> Result<(), ValidationError> {
```

Purpose: This is a lightweight C "lexer" (character-by-character scanner) that catches obvious structural errors BEFORE the full AST parser runs. It's like checking if a book has matching quotation marks and parentheses before trying to read it. **Note:** `fn` without `pub` means this is a PRIVATE function — only callable within this file, not from outside.

State Variables (Lines 108-118)

```
let mut brace_depth = 0i32;
let mut paren_depth = 0i32;
let mut in_string = false;
let mut in_char = false;
let mut in_line_comment = false;
let mut in_block_comment = false;
let mut escaped = false;

let chars: Vec<char> = content.chars().collect();
let len = chars.len();
let mut i = 0;
```

Variable	Type	Purpose
<code>`brace_depth`</code>	<code>`i32`</code>	Counts how many <code>`{`</code> have been opened but not yet closed. Starts at 0. Goes up on <code>`{`</code> , down on <code>`}`</code> .
<code>`paren_depth`</code>	<code>`i32`</code>	Same for parentheses <code>`(`</code> and <code>`)`</code> .
<code>`in_string`</code>	<code>`bool`</code>	Are we currently inside a "string literal"? If yes, <code>`{`</code> and <code>`}`</code> inside should be ignored.
<code>`in_char`</code>	<code>`bool`</code>	Are we inside a <code>`c`</code> character literal? E.g., <code>`{`</code> is NOT an open brace.
<code>`in_line_comment`</code>	<code>`bool`</code>	Are we after <code>`//`</code> ? Everything until newline is a comment.
<code>`in_block_comment`</code>	<code>`bool`</code>	Are we between <code>`/*`</code> and <code>`*/`</code> ? Everything inside is a comment.

Variable	Type	Purpose
<code>`escaped`</code>	<code>`bool`</code>	Was the previous character a backslash <code>`\`</code> ? If so, the NEXT character is escaped (e.g., <code>`\"`</code> inside a string is not a closing quote).
<code>`chars: Vec`</code>	Vector	All characters collected into a random-access array so we can look ahead one character (<code>`chars[i+1]`</code>).
<code>`i`</code>	<code>`usize`</code>	The current position in the character array. The main loop advances <code>`i`</code> .

The Main Scanning Loop (Lines 120-214)

```
while i < len {
    let c = chars[i];
    let next = if i + 1 < len { Some(chars[i + 1]) } else { None };
}
```

Line	Code	Explanation
<code>`while i < len`</code>	Loop condition	Keep scanning characters until we've processed every character in the file.
<code>`let c = chars[i]`</code>	Current character	The character at position <code>`i`</code> .
<code>`let next = if i + 1 < len { Some(chars[i + 1]) } else { None }`</code>	Look-ahead	Peek at the NEXT character without consuming it. Used to detect two-character sequences like <code>`//`</code> and <code>`/*`</code> . Returns <code>`Option`</code> .

State Machine Logic (How each state is handled)

Inside a line comment:

```
if in_line_comment {
    if c == '\n' {
        in_line_comment = false;
    }
    i += 1;
    continue;
}
```

- If we're in a `//` comment, skip everything until we hit a newline `\n`.
- `continue` jumps back to the top of the `while` loop immediately.

Inside a block comment:

```

if in_block_comment {
    if c == '*' && next == Some('/') {
        in_block_comment = false;
        i += 2;    // Skip both * and /
        continue;
    }
    i += 1;
    continue;
}

```

- If we're inside `/* ... */`, look for the `*/` closing sequence.
- When found, turn off `in_block_comment` and skip 2 characters.

Inside a string literal:

```

if in_string {
    if escaped {
        escaped = false;    // Previous \ consumed this char
    } else if c == '\\' {
        escaped = true;     // This \ will escape the next char
    } else if c == '"' {
        in_string = false; // Closing quote found
    }
    i += 1;
    continue;
}

```

- Track backslash escapes so `"\"` (escaped quote inside string) is not treated as a closing quote.
- When `in_string` is true, `{`, `}`, `(`, `)` are all ignored — they're just text.

Inside a character literal: (identical logic for single quotes `'...'`)

Detecting comment starts:

```

if c == '/' && next == Some('/') {
    in_line_comment = true;
    i += 2;
    continue;
}
if c == '/' && next == Some('*') {
    in_block_comment = true;
    i += 2;
    continue;
}

```

- Two-character lookahead: detect `//` or `/*` and enter comment mode.

Counting delimiters:

```

match c {
  '{' => brace_depth += 1,
  '}' => {
    brace_depth -= 1;
    if brace_depth < 0 {
      return Err(ValidationError::PreflightLexicalError(
        "Unmatched closing brace '}' found.".to_string(),
      ));
    }
  }
  '(' => paren_depth += 1,
  ')' => {
    paren_depth -= 1;
    if paren_depth < 0 {
      return Err(ValidationError::PreflightLexicalError(
        "Unmatched closing parenthesis ')' found.".to_string(),
      ));
    }
  }
  _ => {}
}

```

Case	Explanation
`{' => brace_depth += 1`	Opening brace — go one level deeper.
`}' => brace_depth -= 1`	Closing brace — come back up one level.
`if brace_depth < 0`	If depth goes negative, we got a `}` with no matching `{` before it — error!
`_ => {}`	All other characters — do nothing. The `_` is a wildcard that matches anything not already matched.

Final Checks After the Loop (Lines 216-237)

```

if in_string {
    return Err(ValidationError::PreflightLexicalError(
        "Unterminated string literal detected.".to_string(),
    ));
}
if in_block_comment {
    return Err(ValidationError::PreflightLexicalError(
        "Unterminated block comment detected.".to_string(),
    ));
}
if brace_depth > 0 {
    return Err(ValidationError::PreflightLexicalError(
        format!("Unclosed curly brace '{{' ({} missing '}}')", brace_depth),
    ));
}
if paren_depth > 0 {
    return Err(ValidationError::PreflightLexicalError(
        format!("Unclosed parenthesis '(' ({} missing ')'", paren_depth),
    ));
}
Ok(())

```

Check	Meaning
<code>`if in_string`</code>	We reached the end of the file while still inside a string <code>"..."</code> — error!
<code>`if in_block_comment`</code>	We reached the end while still inside <code>`/* ...`</code> — error!
<code>`if brace_depth > 0`</code>	More <code>`{`</code> than <code>`}`</code> — some braces were never closed. <code>`brace_depth`</code> tells us exactly how many.
<code>`if paren_depth > 0`</code>	More <code>`(`</code> than <code>`)`</code> .
<code>`Ok(())`</code>	All checks passed! Return <code>`Ok`</code> with an empty tuple <code>`()`</code> (the <code>`()`</code> means "no value — just success").

Why `{{ and }}` in format strings? In Rust's `format!()` macro, `{` and `}` are special (they mark placeholders). To print a literal brace character, you must double it: `{{` prints `{` and `}}` prints `}`.

Unit Tests (Lines 240-329)

```

#[cfg(test)]
mod tests {
    use super::*;
    use std::io::Write;

```

Code	Explanation
<code>`#[cfg(test)]`</code>	Attribute that tells Rust: "Only compile this block when running tests (<code>`cargo test`</code>). Don't include it in the normal build."

Code	Explanation
<code>`mod tests { ... }`</code>	A nested module called <code>`tests`</code> — all test functions live here by convention.
<code>`use super::*;`</code>	Import everything from the parent module (the <code>`validate_input`</code> function, the types, etc.). <code>`super`</code> means "one level up".
<code>`use std::io::Write;`</code>	Needed to call <code>`writeln!()`</code> on files in tests.

Test 1: Valid C File (Lines 246-262)

```
#[test]
fn test_valid_c_file() {
    let temp_path = Path::new("temp_valid.c");
    let mut file = fs::File::create(temp_path).unwrap();
    writeln!(file,
        "// Simple C Program\n#include <stdio.h>\nint main() {{ printf(\"Hello, World!\"); ret
    }).unwrap();

    let result = validate_input(temp_path);
    let _ = fs::remove_file(temp_path);

    assert!(result.is_ok());
    let validated = result.unwrap();
    assert_eq!(validated.line_count, 3);
    assert!(validated.byte_size > 0);
}
```

Line	Explanation
<code>`#[test]`</code>	Marks this function as a test. <code>`cargo test`</code> will find and run it automatically.
<code>`fs::File::create(temp_path).unwrap()`</code>	Creates a temporary test file. <code>`unwrap()`</code> panics if creation fails — acceptable in tests.
<code>`writeln!(file, ...)`</code>	Writes C code to the temp file.
<code>`let result = validate_input(temp_path)`</code>	Run the function under test.
<code>`let _ = fs::remove_file(temp_path)`</code>	Cleanup: Delete the temp file regardless of test outcome. <code>`let _`</code> discards the result.
<code>`assert!(result.is_ok())`</code>	Assertion: The validation MUST succeed. If it fails, the test fails.
<code>`assert_eq!(validated.line_count, 3)`</code>	Checks that exactly 3 lines were counted.

Test 2: Non-Existent File (Lines 264-269)

```
#[test]
fn test_non_existent_file() {
    let path = Path::new("does_not_exist.c");
    let result = validate_input(path);
    assert!(matches!(result, Err(ValidationError::FileNotFound(_))));
}
```

Code	Explanation
<code>Path::new("does_not_exist.c")</code>	No file is created — this path intentionally doesn't exist.
<code>matches!(result, Err(ValidationError::FileNotFound(_)))</code>	The <code>matches!</code> macro checks if <code>result</code> matches the given pattern. <code>_` is a wildcard matching any string inside. Returns <code>true` if it matches.</code></code>

Test 3: Invalid Extension (Lines 271-281)

Creates a `.txt` file and expects `InvalidExtension` error.

Test 4: Empty File (Lines 283-292)

```
let _file = fs::File::create(temp_path).unwrap();
```

- Creates the file but writes NOTHING to it.
- `let _file =` keeps the file handle alive so it's not deleted before `validate_input` runs. The `_` prefix suppresses the "unused variable" warning.

Test 5: Binary File (Lines 294-304)

```
file.write_all(&[0x7f, 0x45, 0x4c, 0x46, 0x00, 0x01]).unwrap();
```

- Writes raw bytes `7f 45 4c 46 00 01` — the first bytes of an ELF binary (Linux executable).
- The null byte `0x00` at position 4 triggers `InvalidEncoding`.

Test 6: Unmatched Brace (Lines 306-316)

```
writeln!(file, "int main() {{ return 0;}).unwrap();
```

- Writes `int main() { return 0;` — the opening `{` is never closed.
- Expects `PreflightLexicalError`.
- Note: `{{` in the format string prints a single `{`.

Test 7: Unterminated String (Lines 318-328)

```
writeln!(file, "char *s = \"hello;").unwrap();
```

- Writes `char *s = "hello;` — a string that starts with `"` but never ends.
- Expects `PreflightLexicalError` with "Unterminated string literal".

Complete Validation Flow Summary

```
validate_input(path)
  |
  v
Step 1: Does path exist?      No --> FileNotFound
  |
  v
Step 2: Is extension .c or .h? No --> InvalidExtension
  |
  v
Step 3: Can we read the bytes? No --> FileNotReadable
      Does it contain null bytes? Yes --> InvalidEncoding
  |
  v
Step 4: Is it valid UTF-8?    No --> InvalidEncoding
  |
  v
Step 5: Is content non-empty? No --> EmptyFile
  |
  v
Step 6: Balanced braces/parens? No --> PreflightLexicalError
      Open strings or comments?
  |
  v
All checks passed!
  |
  v
Ok(ValidatedCSource { file_path, raw_content, line_count, byte_size })
```

Key Rust Concepts Introduced in This Module

Concept	Example in This File	What it means	
Result	<code>-> Result</code>	Either success <code>Ok(T)</code> or failure <code>Err(E)</code> .	
Option operator	<code>fs::read(file_path)?</code>	Automatically returns <code>Err</code> upward if result is an error.	
Early return	<code>return Err(...)</code>	Exit the function immediately with an error.	
Pattern match	<code>if let Some(ext) = file_path.extension()</code>	Pattern-match to extract a value from <code>Option</code> .	

Concept	Example in This File	What it means	
<code>map_err(\</code>	<code>e\</code>	<code>...)\`</code>	Converts one error type to another using a closure.
<code>#[test]\`</code>	Test-only code block, excluded from release builds.		
<code>test]\`</code>	Marks a function as a unit test for `cargo test`.		
<code>assert!()\` / `assert_eq!()\`</code>	Test assertion macros — panic if condition is false.		
<code>matches!(expr, pattern)\`</code>	Tests if an expression matches a given pattern.		
Parser machine	<code>`brace_depth`, `in_string`, etc.</code>	Track parser state character-by-character.	

Summary

Module 1 is a **six-step validation gatekeeper** that ensures only valid, readable, correctly-formed C source files enter the pipeline:

- File exists on disk
- Extension is `.c` or `.h`
- File is readable (no permission issues, no binary content)
- Content is valid UTF-8 text
- Content is non-empty
- Braces, parentheses, strings, and comments are balanced

It is the **only module** that interacts directly with the operating system's file system. All subsequent modules work on the `ValidatedCSource` struct that Module 1 produces — they never re-read the file from disk.

The **7 unit tests** cover every possible failure path plus the happy path, forming a complete correctness specification for the validation logic.

Document generated for C2SafeRust-LLM Mini Project S3

Module 1: Input Validation — Complete Line-by-Line Explanation